

STUDENT RECORD & ACADEMIC MANAGEMENT SYSTEM

Python Unit 2 • Essential Assignment Content

Only the important work, deliverables and submission steps

1. What You Actually Have to Make

Your final project is a **console-based Student Record and Academic Management System**. The Library Management System in the first document is only an example.

The five required Python collections are:

Data structure	Use it for
String	Student name, department, address, email
List	Student records, subjects, marks, attendance
Tuple	Roll number, registration number, date of birth
Set	Departments, subjects, student clubs
Dictionary	Complete student profile

Student profile fields: **Roll Number, Name, Department, Subjects, Marks, Attendance and Contact Information**.

Main menu/features:

Option	Required function
1. Add Student	Add record; reject duplicate roll number
2. Search Student	Search by roll number
3. Update Student	Update selected information
4. Delete Student	Delete a record
5. Display Records	Show all students
6. Calculate Average	Calculate average marks
7. Highest Scorer	Find highest scorer
8. By Department	List students by department
9. Count Students	Count records
10. Generate Reports	Show useful totals and unique collections
0. Exit	Close program

2. Exact Order of Work

Follow this order. The assignment is not only about writing the final Python code.

Step	What you make
1. Problem Understanding	Inputs, outputs, required data, relationships, suitable structure
2. Data Structure Selection	Why String/List/Tuple/Set/Dictionary is appropriate
3. Algorithms	Plain-English numbered steps
4. Flowcharts	Proper diagrams for required operations
5. Pseudocode	Structured English matching each flowchart
6. Python Implementation	Collection exercises + integrated system
7. Testing	Normal, invalid, duplicate, empty and boundary tests
8. Documentation	Organize the work, observations and screenshots
9. GitHub	Repository + README + project files

Do the design work **before** the complete implementation: algorithm → flowchart → pseudocode → code.

3. Files / Sections You Need

File / section	Important content
problem_analysis	8 problems: inputs, outputs, data, structure, processing
data_structure_selection	Reason for each collection
algorithms	8 required algorithms
flowcharts	7 required flowcharts
pseudocode	Pseudocode for major operations
string_programs.py	Required string methods
list_programs.py	Required list methods
tuple_programs.py	Tuple operations
set_programs.py	Set operations
dictionary_programs.py	Dictionary operations
integrated_system.py	Complete student management system
testing	Test cases + results + observations
screenshots	Actual execution screenshots
README.md	Purpose, features, structures, run instructions
GitHub	All files organized and uploaded

4. Eight Required Algorithms

1. **Add Student** — Read details → check duplicate roll → reject if duplicate → create record → store → confirm.
2. **Search Student** — Read roll → check records → display matching record → otherwise show not found.
3. **Update Record** — Read roll → find record → choose field → read new value → update → confirm.
4. **Delete Record** — Read roll → find record → remove if found → confirm; otherwise not found.
5. **Calculate Average** — Find student → read marks → total → divide by number of marks → display average.
6. **Find Highest Scorer** — Check empty collection → calculate each student's score/average → keep highest → display.
7. **List by Department** — Read department → compare all records → collect matches → display.
8. **Count Students** — Count student records → display total.

5. Flowcharts You Need

Flowchart	Required logic
Add Student	START → Read details → Duplicate? → reject/store → success → STOP
Search Student	START → Read roll → loop → Match? → display/continue → not found → STOP
Update Record	START → Read roll → Found? → read field → update → success → STOP
Delete Record	START → Read roll → Found? → remove → success → STOP
Display Records	START → Empty? → no records / loop and display all → STOP
Calculate Average	START → Read roll → Found? → read marks → calculate → display → STOP
Generate Reports	START → Empty? → unique departments/subjects + totals + highest scorer → display → STOP

Use standard symbols: **rounded rectangle = Start/Stop**, **rectangle = Process**, **diamond = Decision**, **parallelogram = Input/Output**.

6. Pseudocode Pattern

```
START
  READ roll number
  SET found = FALSE

  FOR EACH student IN collection
    IF roll number matches THEN
      DISPLAY student record
      SET found = TRUE
      EXIT LOOP
    END IF
  END FOR

  IF found = FALSE THEN
    DISPLAY "Student Not Found"
  END IF
STOP
```

Repeat this style for all major operations. Do not copy Python syntax and call it pseudocode.

7. Collection Programs: What to Demonstrate

Collection	Must demonstrate
Strings	indexing, slicing, concatenation, f-strings, upper, lower, title, capitalize, strip, split, join, replace, find, count, startswith, endswith
Lists	creation, indexing, traversal, append, insert, extend, remove, pop, clear, index, count, sort, reverse, list comprehension
Tuples	creation, packing, unpacking, indexing, count, index, immutability
Sets	creation, add, remove, discard, membership, union, intersection, difference, duplicate removal
Dictionaries	creation, access, keys, values, items, get, update, pop, popitem, clear

Keep the examples student-related. Avoid unnecessary databases, GUI, pandas, NumPy or advanced OOP.

8. Testing: What You Need to Show

Test	Example
Normal	Add and search an existing student
Duplicate	Add an existing roll number
Invalid	Search unknown roll / invalid menu choice
Empty	Run display/report/search with no records
Boundary	Marks 0 and 100; one student
Update	Change student information
Delete	Delete existing and non-existing student
Dry run	Trace one representative operation step by step

Testing table columns: **Test ID | Operation | Input | Expected Output | Actual Output | Result | Observation**

9. GitHub + README: Actual Steps

The assignment requires an updated GitHub repository. Treat this as a final submission step.

Step 1: Create repository

Create a repository such as **Student-Record-Management-System**.

Step 2: Create the structure

```
Student-Record-Management-System/  
■■■ README.md  
■■■ integrated_system.py  
■■■ string_programs.py  
■■■ list_programs.py  
■■■ tuple_programs.py  
■■■ set_programs.py  
■■■ dictionary_programs.py  
■■■ documentation/  
■■■ flowcharts/  
■■■ screenshots/
```

Step 3: Make README.md

README section	What to write
Project Title	Student Record and Academic Management System
Objective	What the project demonstrates
Features	Add, search, update, delete, display, average, highest scorer, department, reports
Data Structures	Five structures and their uses
How to Run	Python version + command to run integrated_system.py
Testing	Short description of tests
Author	Your name / class / roll if required

Step 4: Upload — Upload Python files, documentation, flowcharts and screenshots.

Step 5: Commit — Use clear messages such as “Add collection programs”, “Add algorithms and pseudocode”, “Add integrated system”, “Add testing”, “Final submission”.

Step 6: Final GitHub check — Open the repository like a teacher. Check that README, code, flowcharts and screenshots are visible.

10. Final Submission Checklist

✓	Requirement
■	Problem analysis completed
■	Data structure selection explained
■	8 algorithms completed
■	7 required flowcharts completed
■	Pseudocode completed
■	String methods demonstrated
■	List methods demonstrated
■	Tuple operations demonstrated
■	Set operations demonstrated
■	Dictionary methods demonstrated
■	Integrated system works
■	Duplicate roll number handled
■	Invalid input handled
■	Empty collection handled
■	Testing table completed
■	Dry-run / trace completed
■	Actual screenshots captured
■	README.md completed
■	GitHub repository created and organized
■	Final code tested

The whole assignment in one line

Understand → select data structures → write algorithms → draw flowcharts → write pseudocode → code → test → document → add screenshots → create README → upload to GitHub → final check.